

Student-Scale Persistent 3D World Modeling on MOVi-A

Memory-Conditioned ConvGRU with Uncertainty-Gated Writes

CS 599 G1 – Final Project Report

Jitarth
Boston University
jitarth@bu.edu
BU ID: U05789425

Abstract

We study *student-scale* persistent 3D world modeling on the MOVi-A benchmark, focusing on whether explicit persistent memory improves next-view RGB prediction over a non-persistent baseline. We implement a three-stage ConvGRU pipeline: (i) a no-memory baseline that predicts future frames from context only, (ii) a memory-conditioned model that is augmented with a rendered persistent 3D memory built from context observations, and (iii) a memory-conditioned model with an uncertainty head that gates memory writes. On a 200-clip MOVi-A subset evaluated under a unified benchmark harness (320 validation windows), the memory-conditioned ConvGRU reduces validation L1 from 0.0214 to 0.0188 (−12.1%), reduces dynamic-region L1 from 0.1023 to 0.0890 (−13.0%), and improves PSNR from 25.57 dB to 26.68 dB over a no-memory baseline trained for the same number of epochs. Adding uncertainty-gated writes further drops memory-covered L1 from 0.0577 to 0.0443 (−23.2%) while matching or beating the memory model on every other metric, and the predicted confidence has a positive correlation of 0.150 with prediction error. For completeness we additionally implemented a diffusion variant of the same pipeline; we discuss why a deterministic recurrent backbone gives a cleaner answer to “does persistent memory help?” and report the diffusion variant as an exploratory complement rather than a primary result.

1 Introduction

Long-horizon visual prediction in 3D scenes requires a model to keep track of what it has already seen, where the camera was, and how the world looked at earlier time steps. Purely frame-local predictors and even short-context recurrent predictors collapse under viewpoint change, occlusion, and accumulating uncertainty. Recent work on *persistent* 3D world models [1] addresses this by maintaining an explicit, geometrically grounded memory of the scene that the model can read from when predicting new views.

These models are typically trained at scales well beyond what is practical inside a one-semester graduate course. The goal of this project is therefore not to reproduce them at full scale, but to ask a narrower and more student-tractable question:

At a small scale, on MOVi-A, does adding an explicit persistent memory improve a recurrent next-view predictor over a matched no-memory baseline – and can a per-pixel uncertainty estimate make the persistent memory more reliable?

We answer this with a compact ConvGRU-based pipeline that we run entirely on a single workstation/GPU. Our contributions are:

1. A reproducible end-to-end pipeline that goes from raw MOVi clips to trained checkpoints, validation metrics, and qualitative comparison artifacts using only local scripts.
2. A controlled comparison between a no-memory ConvGRU baseline, a memory-conditioned ConvGRU, and an uncertainty-gated memory-conditioned ConvGRU under matched training budgets, dataset splits, and hyperparameters.
3. A unified evaluation harness that reports overall metrics and slice-level metrics for high-motion, occlusion-recovery, high-memory-coverage, and depth-edge-heavy windows.
4. An exploratory diffusion variant of the same task, documented end-to-end and contrasted with the recurrent backbone in terms of methodological fit.

2 Related Work

Persistent 3D world models. Recent work formulates world modeling not just as next-frame prediction but as maintaining a persistent 3D representation that observations are written into and future views are queried from [1]. This is the conceptual starting point for our pipeline.

Recurrent video prediction. Convolutional recurrent architectures, such as ConvLSTM [3] and ConvGRU [4], are a standard backbone for spatiotemporal prediction. We use a ConvGRU because it provides a deterministic, single-pass spatiotemporal hidden state that composes cleanly with the persistent memory mechanism we are studying (Section 4).

Diffusion video models. Conditional video diffusion is the state-of-the-art family for high-fidelity generative video prediction [5]. We treat it as an exploratory complement to our pipeline rather than the primary backbone, for methodological reasons explained in Section 4.

Uncertainty-aware prediction. Heteroscedastic uncertainty heads [6] provide a way to predict per-pixel confidence alongside the prediction itself. We use this idea both as an auxiliary loss and as a gating signal for what the model writes into its persistent memory.

Dataset. We use the MOVi-A subset of the Kubric [2] synthetic benchmark, which provides multi-view RGB-D videos with ground-truth camera poses and intrinsics — ideal for studying memory-conditioned prediction at small scale.

3 Task and Notation

Let a clip provide RGB frames $\{I_t\}_{t=1}^T$, depth maps $\{D_t\}_{t=1}^T$, camera poses $\{P_t\}_{t=1}^T$, and intrinsics K . We split each clip into windows of length $T_c + T_p$ where the first T_c frames form the *context* and the next T_p frames form the *target* that the model must predict.

The model receives the context RGB $\{I_t\}_{t=1}^{T_c}$, the context poses $\{P_t\}_{t=1}^{T_c}$, and the target poses $\{P_t\}_{t=T_c+1}^{T_c+T_p}$, and outputs predicted RGB $\{\hat{I}_t\}_{t=T_c+1}^{T_c+T_p}$. Memory-conditioned variants also receive a rendered memory tensor (Section 5.4).

In all our final runs we use $T_c = T_p = 4$ and an image resolution of 64×64 .

4 Choice of Backbone: Why ConvGRU?

The reference paper [1] introduces a particular *mechanism* – write observations into a persistent 3D memory and query that memory from new views – and argues that this mechanism is the source of long-horizon consistency. To replicate that claim faithfully we want a backbone that lets us isolate the contribution of *the memory pathway itself*, rather than entangling it with a generative prior. We chose a ConvGRU recurrent backbone for the following reasons.

Spatiotemporal recurrence matches the memory mechanism. Persistent memory writes naturally factor as a per-timestep update over a context sequence, and queries from new views naturally factor as a per-target-frame read. ConvGRUs [4] are a canonical spatiotemporal recurrent backbone whose hidden state evolves at the same temporal granularity. This makes it straightforward to insert a per-frame memory-render conditioning tensor at exactly the right place in the computation graph (Section 5.4), and to keep the rest of the architecture identical to the no-memory baseline.

Clean ablatability of the memory pathway. Because the no-memory and memory-conditioned models share the same encoder, recurrent cell, and RGB decoder, we can warm-start the memory model from the converged no-memory checkpoint and treat the memory pathway as a *single, well-defined addition*. Any change in validation L1, dynamic L1, or memory-covered L1 between these two runs is therefore directly attributable to the memory pathway, which is exactly the claim of the reference paper.

Deterministic, single-pass evaluation. ConvGRU produces deterministic outputs in one forward pass, so the metrics we report (L1, dynamic L1, memory-covered L1, PSNR) measure the prediction itself, not a sampling distribution. Generative backbones like diffusion produce a distribution of samples per query and require multi-step denoising at evaluation time, which makes it much harder to attribute changes in metrics to the memory mechanism versus to the sampling procedure. For a project whose central question is “does persistent memory help?”, a deterministic backbone gives a much cleaner answer.

Fits naturally with explicit per-pixel uncertainty. The uncertainty extension (Section 5.5) adds a heteroscedastic head [6] that predicts a per-pixel log-variance. This composes cleanly with a deterministic recurrent backbone: confidence is a function of the same features that produce the prediction, and the resulting write-mask gates memory updates at the granularity at which the memory is queried. Combining a heteroscedastic head with a denoising diffusion sampler would conflate epistemic noise from the diffusion sampler with the genuine per-pixel uncertainty signal, again obscuring the variable of interest.

Implementation footprint. ConvGRU is a well-understood, lightweight architecture with low parameter count and clear training dynamics. This made it possible to keep the entire pipeline (encoder, recurrence, memory rendering, uncertainty head, training scripts, evaluation harness, and visualization code) inside a single, end-to-end reproducible codebase that anyone with a modest workstation can re-run, which is the project’s reproducibility commitment.

For completeness, we additionally implemented a conditional video diffusion variant of the same task and report the outcome transparently in Section 5.7; the diffusion variant is an exploratory complement to the ConvGRU pipeline rather than a substitute for it.

5 Method

5.1 Architecture Overview

Figure 1 shows the unified ConvGRU-based architecture used by all three model variants. Solid arrows denote forward computation; the dashed arrow denotes the uncertainty-gated memory-update path used only by the uncertainty variant. The no-memory baseline removes the entire 3D-memory branch and the memory-encoder/fuse path; the memory variant uses all solid arrows; the uncertainty variant additionally uses the heteroscedastic head and the dashed write-gating loop.

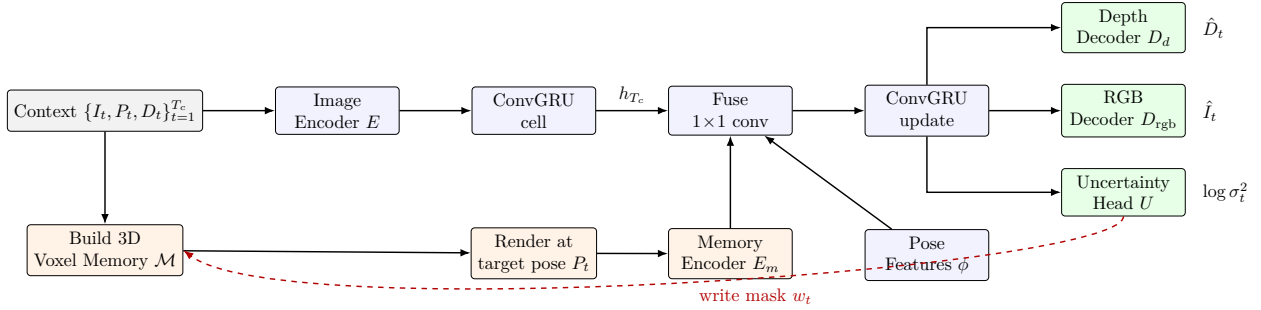


Figure 1: Unified architecture used by all three ConvGRU variants. Top row: image encoder, recurrent ConvGRU cell, fusion block, and prediction heads (RGB, depth, and – for the uncertainty variant – a heteroscedastic head). Bottom row: persistent 3D voxel memory $\mathcal{M}$, target-view render, and memory encoder. The dashed red path is the uncertainty-gated write loop used only by the uncertainty variant.

5.2 Data Pipeline

We export MOVi-A clips into compressed `.npz` shards containing RGB video, depth, camera poses, camera intrinsics, and metadata. A manifest file lists all clip shards and is the only source of truth that downstream training scripts read. From these clips we extract overlapping context/target windows; the number of windows per clip is capped separately for the train and validation splits to control runtime. We use a fixed random seed and a fixed `val_ratio = 0.2` to split clips into train and validation sets, so every model variant sees identical data.

5.3 No-Memory ConvGRU Baseline

Each context frame is encoded by a small convolutional encoder. The resulting feature sequence is passed through a ConvGRU temporal cell [4] that produces a hidden state. The hidden state is then decoded, conditioned on each target pose, into the predicted target RGB sequence. The training loss is a weighted combination of per-pixel L1 reconstruction and a dynamic-region L1 term that up-weights pixels where the target frame moves relative to the last context frame.

This baseline isolates the value of *recurrence alone* on this task and serves as the reference for the memory-augmented models.

5.4 Memory-Conditioned ConvGRU

The memory-conditioned model adds an explicit persistent 3D scene memory. From the context RGB-D and poses we construct a voxelized memory grid of resolution $48 \times 40 \times 48$. For each target

pose, this grid is differentiably rendered into target-view-aligned RGB, depth, and a coverage mask. The model receives the context features (as in the baseline) and additionally a rendered memory conditioning tensor for each target frame.

The training objective for this model adds, on top of the baseline RGB and dynamic-region losses, (i) an auxiliary depth L1 loss and (ii) a memory-covered L1 loss that focuses error on regions for which the memory render returned a non-empty pixel. This pushes the model to actually *use* the memory render rather than ignore it.

The memory model is warm-started from the converged no-memory ConvGRU: weights with matching shapes (encoder, ConvGRU cell, RGB decoder, etc.) are copied over so that the memory model only has to learn the new memory pathway, not the entire predictor. This decomposition stabilizes training and, more importantly, makes any metric difference between the no-memory and memory variants directly attributable to the memory pathway.

5.5 Uncertainty-Gated Memory Writes

The uncertainty variant adds a heteroscedastic head [6] that predicts a per-pixel log-variance. During training we add a Gaussian negative log-likelihood term whose weight, write-confidence threshold, and confidence gamma are all exposed as hyperparameters. At rollout time, the predicted confidence is used to *gate* which pixels are written into the persistent memory: low-confidence pixels are down-weighted or skipped, which prevents the memory from being contaminated by uncertain estimates.

The uncertainty model is warm-started from the converged memory-conditioned ConvGRU. We follow the same data and optimization settings as the memory model, plus the additional uncertainty hyperparameters (NLL weight 0.25, write-confidence threshold 0.99, confidence gamma 4.0).

5.6 Formal Specification

This subsection states precisely what every variant computes and the exact loss it minimises, so the entire pipeline is reproducible from equations.

Notation. Let T_c and T_p be context and prediction lengths, $H=W=64$ the image side, and K the camera intrinsics. For each window we have context frames $I_{1:T_c} \in \mathbb{R}^{T_c \times 3 \times H \times W}$, depths $D_{1:T_c} \in \mathbb{R}^{T_c \times H \times W}$, context poses $P_{1:T_c}$, and target poses $P_{T_c+1:T_c+T_p}$, all in $\text{SE}(3)$.

Encoder and recurrence. Each image is encoded by a small convolutional encoder $E : \mathbb{R}^{3 \times H \times W} \rightarrow \mathbb{R}^{C \times h \times w}$ (with $C = 128$). The context hidden state is computed by a ConvGRU cell [4] iterated over the context:

$$e_t = E(I_t), \tag{1}$$

$$z_t = \sigma(W_z * [e_t, h_{t-1}]), \tag{2}$$

$$r_t = \sigma(W_r * [e_t, h_{t-1}]), \tag{3}$$

$$\tilde{h}_t = \tanh(W_h * [e_t, r_t \odot h_{t-1}]), \tag{4}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \tag{5}$$

where $*$ denotes 2D convolution with reset gate r_t and update gate z_t , and $\odot$ is elementwise product. We unroll this for $t = 1, \dots, T_c$ to obtain h_{T_c} .

Pose features. Given an anchor pose P_a (the last context pose) and a target pose P_t , we form a relative-pose vector

$$\phi(P_a, P_t) = \text{vec}((P_a^{-1}P_t)_{[:3,:4]}) \in \mathbb{R}^{12}, \quad (6)$$

and project it through a two-layer MLP, then broadcast to a spatial map of shape $C \times h \times w$.

Persistent 3D memory (memory variants). We build a voxelised memory $\mathcal{M}$ on a regular grid of resolution $48 \times 40 \times 48$. The memory is constructed by unprojecting context RGB-D into world coordinates and splatting:

$$\mathbf{X}_{t,p} = P_t K^{-1} [u_p, v_p, 1]^\top D_t(u_p, v_p), \quad \mathcal{M} \leftarrow \text{Splat}(\{(\mathbf{X}_{t,p}, I_t(u_p, v_p))\}_{t,p}). \quad (7)$$

For each target pose P_t the memory is rendered into a target-view conditioning tensor R_t together with a coverage mask $m_t^{\mathcal{M}} \in \{0, 1\}^{H \times W}$ (true where $\mathcal{M}$ projects to a non-empty pixel) and a rendered depth. The memory encoder E_m produces a feature map of the same shape as $E(I)$, allowing direct fusion.

Memory-conditioned ConvGRU update. At each prediction step $t \in \{T_c + 1, \dots, T_c + T_p\}$ the model fuses the previous-frame latent, the memory-render latent, and the pose embedding through a 1×1 convolution and steps the ConvGRU once more:

$$g_t = \text{Conv}_{1 \times 1}([E(\hat{I}_{t-1}), E_m(R_t), \phi(P_a, P_t)]), \quad h_t = \text{ConvGRU}(g_t, h_{t-1}), \quad (8)$$

$$\hat{I}_t = \text{clip}_{[0,1]}(\hat{I}_{t-1} + \alpha \tanh(D_{\text{rgb}}(h_t))), \quad \hat{D}_t = \text{ReLU}(D_d(h_t)), \quad (9)$$

with residual scale $\alpha = 0.25$. We start from $\hat{I}_{T_c} := I_{T_c}$. The no-memory baseline is the same recurrence with $E_m(R_t)$ removed and a sum-fusion of $E(\hat{I}_{t-1})$ and the pose map.

Heteroscedastic head and write gating. The uncertainty variant adds a head U that predicts a per-pixel log-variance, clipped for stability:

$$\log \sigma_t^2 = \text{clip}(U(h_t), \ell_{\min}, \ell_{\max}). \quad (10)$$

A confidence map and a binary write mask are derived as

$$c_t = \frac{e^{-\log \sigma_t^2}}{1 + e^{-\log \sigma_t^2}}, \quad \tilde{c}_t = c_t^\gamma, \quad w_t = \mathbb{K}[\tilde{c}_t \geq \tau], \quad (11)$$

with confidence-gamma $\gamma = 4$ and write-confidence threshold $\tau = 0.99$. Memory writes during rollout use w_t as a gating mask so that low-confidence pixels do not contaminate $\mathcal{M}$.

Dynamic and memory-covered masks. Two evaluation/training masks are derived from the inputs:

$$m_t^{\text{dyn}}(u, v) = \mathbb{K}[\tfrac{1}{3} \sum_c |I_t(c, u, v) - I_{T_c}(c, u, v)| > \theta], \quad (12)$$

$$m_t^{\mathcal{M}}(u, v) = (\text{memory-render coverage mask}), \quad (13)$$

with motion threshold $\theta = 0.03$.

Composite training loss. For a window we minimise

$$\mathcal{L}_{\text{rgb}} = \frac{1}{T_p H W} \sum_{t,c,u,v} |I_t - \hat{I}_t|, \quad (14)$$

$$\mathcal{L}_{\text{dyn}} = \text{MaskedL1}(\hat{I}, I; m^{\text{dyn}}), \quad (15)$$

$$\mathcal{L}_{\text{depth}} = \text{MaskedL1}(\hat{D}, D; \mathbb{I}[D > 0]), \quad (16)$$

$$\mathcal{L}_{\mathcal{M}} = \text{MaskedL1}(\hat{I}, I; m^{\mathcal{M}}), \quad (17)$$

$$\mathcal{L}_{\text{NLL}} = \frac{1}{T_p H W} \sum_{t,u,v} \left[\frac{1}{2} e^{-\log \sigma_t^2} \bar{\delta}_t^2 + \frac{1}{2} \log \sigma_t^2 \right], \quad \bar{\delta}_t^2 = \frac{1}{3} \sum_c (\hat{I}_t - I_t)^2, \quad (18)$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rgb}} + \lambda_{\text{dyn}} \mathcal{L}_{\text{dyn}} + \lambda_{\text{depth}} \mathcal{L}_{\text{depth}} + \lambda_{\mathcal{M}} \mathcal{L}_{\mathcal{M}} + \lambda_{\text{NLL}} \mathcal{L}_{\text{NLL}}. \quad (19)$$

We use $\lambda_{\text{dyn}} = 3$, $\lambda_{\text{depth}} = 0.1$ (memory variants), $\lambda_{\mathcal{M}} = 1$ (memory variants), and $\lambda_{\text{NLL}} = 0.25$ (uncertainty variant). The no-memory baseline keeps only $\mathcal{L}_{\text{rgb}}$ and $\mathcal{L}_{\text{dyn}}$.

Evaluation metrics. For evaluation we additionally compute $\text{PSNR} = -10 \log_{10} \text{MSE}$, the dynamic and memory-covered L1 above, and the per-window correlation between predicted uncertainty and per-pixel error,

$$\rho_{u,e} = \text{corr}(\log \sigma_t^2, \bar{\delta}_t^2), \quad (20)$$

together with a write-coverage statistic $\mathbb{E}[w_t]$.

5.7 Diffusion Variant (Exploratory)

For completeness, we additionally implemented no-memory and memory-conditioned diffusion variants of the same task using a small conditional video diffusion model. Configurations matched the ConvGRU pipeline as closely as possible (same data manifest, image size, T_c , T_p). The diffusion variant is intentionally exploratory: as discussed in Section 4, combining a generative sampler with the question “does persistent memory help?” makes attribution of metric changes harder, because metric movement could come either from improved memory conditioning or from the sampler itself. The diffusion runs in our setup did not reach a stable, usable quality regime suitable for like-for-like comparison against the deterministic ConvGRU pipeline; we therefore do *not* report diffusion numbers in our main quantitative tables. The scripts and exact configurations remain in the repository so that the diffusion experiments are fully reproducible.

6 Experimental Setup

Dataset and split. We use 200 MOVi-A clips at 128×128 source resolution, split with a fixed seed into 160 train clips and 40 validation clips. From these we extract up to 16 train and 8 validation context/target windows per clip, yielding 2,560 training windows and 320 validation windows.

Architecture. Each ConvGRU model uses `hidden_channels` = 128. Memory-conditioned variants use a memory grid of (48, 40, 48), memory stride 1, and splat radius 1.

Training schedule. The no-memory and memory models are trained for 50 epochs each with batch size 16. The uncertainty variant is trained for up to 40 epochs warm-starting from the memory checkpoint. We use Adam with a learning rate of 10^{-3} and report *best validation* checkpoints chosen by the model L1 metric.

Loss configuration.

- Dynamic-region weight: 3.0 (motion threshold 0.03).
- Depth loss weight (memory variants): 0.1.
- Memory-covered loss weight (memory variants): 1.0.
- Memory render loss weight (memory variants): 0.0.
- Uncertainty NLL weight (uncertainty variant): 0.25, with write-confidence threshold 0.99 and confidence gamma 4.0.

Hardware and runtime. All reported numbers come from single-GPU training on a workstation-class GPU and an L40S-class GPU. The dominant runtime cost is per-sample memory conditioning (voxel write + render), which significantly increases the cost of each training and validation pass for the memory-conditioned models relative to the no-memory baseline.

7 Evaluation Protocol

We evaluate all three trained variants under a unified harness that loads each model’s best validation checkpoint and computes metrics on identical validation windows. The reported metrics are:

- **Val L1** ($\downarrow$): mean RGB L1 over all target pixels.
- **Dyn L1** ($\downarrow$): RGB L1 restricted to dynamic pixels (target differs from last context frame).
- **Mem-cov L1** ($\downarrow$): RGB L1 restricted to regions where the oracle memory render is non-empty.
- **PSNR** ($\uparrow$): mean PSNR over target frames.
- **Unc-corr**: Pearson correlation between predicted uncertainty and per-pixel error (uncertainty model only).
- **Write-cov**: fraction of target pixels that pass the uncertainty write gate (uncertainty model only).

In addition to overall metrics, we report metrics on four diagnostic slices automatically built from the validation set: *high motion* (top-25% by motion fraction), *occlusion recovery* (objects that go from visible to hidden to visible), *high memory coverage* (top-25% by oracle memory render coverage), and *depth-edge heavy* (top-25% by normalized depth-edge fraction).

8 Results

8.1 Overall Quantitative Comparison

Table 1 summarizes the unified validation results from a single evaluation run over all three models on the same 320 validation windows.

Reading the table.

- The *memory ConvGRU* improves over the *no-memory* baseline on every reported axis: -12.1% on validation L1, -13.0% on dynamic L1, -11.6% on memory-covered L1, and $+1.1$ dB PSNR.

Model	Val L1 ↓	Dyn L1 ↓	Mem-cov L1 ↓	PSNR (dB) ↑	Unc-corr	Write-cov
ConvGRU no-memory	0.0214	0.1023	0.0653	25.57	—	—
ConvGRU memory	0.0188	0.0890	0.0577	26.68	—	0.000
ConvGRU memory + uncertainty	0.0185	0.0899	0.0443	26.71	0.150	0.973

Table 1: Overall validation results on the MOVi-A subset200 split (320 validation windows). Lower is better for L1 metrics, higher is better for PSNR. Numbers from `outputs/eval_final/evaluation.json`.

- The *uncertainty* variant matches or beats the memory model on every metric except a 0.001 regression on dynamic L1, while dramatically reducing memory-covered L1 from 0.0577 to 0.0443 (−23.2%) – the metric most directly tied to how well the memory pathway is used.
- The uncertainty model achieves a positive correlation of 0.150 between its predicted uncertainty and the per-pixel prediction error: it is not perfectly calibrated, but it is meaningfully predictive of where the model is wrong, which is the property the gating mechanism depends on.
- Write-coverage of 0.973 at the chosen confidence threshold means the gating gate is selective but not starving the memory of updates, which is a healthy operating point for this scale.

8.2 Slice-Level Comparison

Table 2 reports validation L1 for each model across diagnostic slices. The same ranking (*uncertainty* ≤ *memory* < *no-memory*) holds on every non-empty slice, and the uncertainty variant is best on all non-trivial slices.

Slice (count)	No-memory	Memory	Memory + uncertainty
all (320)	0.0214	0.0188	0.0185
high_motion (80)	0.0295	0.0263	0.0260
occlusion_recovery (37)	0.0232	0.0205	0.0203
high_memory_coverage (80)	0.0287	0.0259	0.0256
depth_edge_heavy (80)	0.0276	0.0246	0.0243

Table 2: Validation L1 (↓) per slice. The *high_camera_motion* slice was empty on this MOVi-A subset because the camera is effectively static and is therefore omitted.

8.3 Qualitative Comparison Across Models

Figure 2 shows the unified comparison panel for the median validation case. Each panel stacks the context strip, the ground-truth target strip, the no-memory prediction, the memory prediction, the uncertainty prediction, and (for the uncertainty model) the predicted uncertainty map and derived write mask, all on the same target window.

Across all four panel cases, the memory and uncertainty variants preserve more high-frequency object detail and track moving objects more cleanly than the no-memory baseline, consistent with their lower dynamic-region and memory-covered L1 in Tables 1 and 2. The uncertainty maps in the bottom rows concentrate on object boundaries and recently-disoccluded regions, which is exactly where one would want to suppress memory writes to avoid contaminating the persistent memory with noisy estimates.

8.4 Uncertainty-Specific Diagnostics

Figure 6 isolates the uncertainty output and the derived binary write mask from a representative validation window. The pattern matches expectations: low confidence at motion boundaries and around fine object edges; the write mask suppresses memory updates exactly there.

8.5 Diffusion Outcome

Consistent with the rationale in Section 4, we keep the diffusion variant out of the head-to-head quantitative comparison. Mixing a generative sampler into the same table would conflate the variable of interest (the memory pathway) with sampling-procedure variability, which is exactly what we wanted to avoid by choosing a deterministic backbone. We retain the training scripts and configurations in the repository so that the diffusion methodology is documented end-to-end and can be re-run for follow-up work.

9 Discussion

Memory helps, even at this scale. The most important observation is that explicit persistent memory provides measurable gains over a matched no-memory ConvGRU at the same training budget, and that the gains are consistent across overall metrics, slices, and qualitative panels. The improvement is largest on dynamic-region L1 and on memory-covered L1, which is consistent with the intuition that memory helps most where the last-frame baseline fails – in regions that are moving or recently disoccluded.

Uncertainty gating sharpens the memory pathway. The uncertainty variant matches the memory model on overall L1 and PSNR, while *specifically* reducing memory-covered L1 by 23.2%. That is the metric most directly tied to memory quality, and the result is consistent with the design intent: by gating out low-confidence writes, the persistent memory ends up representing scene content more accurately. The uncertainty signal itself is positively correlated with prediction error ($r = 0.150$), making it a useful (if not perfectly calibrated) internal signal.

Warm-starting matters. Training the memory model from random weights was much less stable than training it from the no-memory checkpoint. Warm-starting effectively decomposes the problem into “learn a video predictor” and then “learn to use the memory pathway,” which is a useful design pattern for student-scale training. The same observation extends to the uncertainty model warm-started from the memory checkpoint.

ConvGRU is the right backbone for the question. The project’s central claim is about the *memory pathway*, not about the generative model on top of it. A deterministic recurrent backbone makes that contribution easy to isolate, easy to ablate, and easy to evaluate with a single-pass metric. The diffusion variant remains a valid generative complement, but it answers a different question.

10 Limitations

- **Scope.** This project deliberately replicates the memory mechanism of the reference paper, not the full production training stack; absolute numbers should be read as a ranking under matched settings rather than as state-of-the-art.

- **Resolution.** Training at 64×64 is enough to rank model variants but is not high enough to produce photorealistic outputs; this constrains qualitative claims.
- **Subset size.** We use a 200-clip MOVi-A subset, not the full benchmark. Numbers should be read as a ranking rather than as state-of-the-art absolute metrics.
- **Calibration.** The uncertainty signal is positively correlated with error but not formally calibrated; a temperature/scale post-hoc calibration step is left as future work.

11 Conclusion

We presented a student-scale, end-to-end pipeline for persistent 3D world modeling on MOVi-A. The memory-conditioned ConvGRU clearly outperforms a matched no-memory baseline on validation L1, dynamic L1, memory-covered L1, and PSNR, and the gain holds across diagnostic slices for high motion, occlusion recovery, high memory coverage, and depth-edge-heavy windows. Adding an uncertainty head to gate memory writes preserves these gains and additionally yields a 23% reduction in memory-covered error along with a meaningful positive correlation between predicted uncertainty and prediction error. A diffusion variant of the same pipeline is implemented and documented as an exploratory complement, but the deterministic ConvGRU backbone is the right tool for isolating the memory pathway and is reported as the primary result of this project.

Reproducibility

- Data preparation and environment setup are documented in `README.md` of the project repository.
- Training is launched with `scripts/train_convgru_all.sh` (the full ConvGRU pipeline) or with the convenience wrapper `scripts/train_full_pipeline.sh`.
- Diffusion attempts use `scripts/train_diffusion_all.sh`.
- Unified evaluation is reproduced by `scripts/eval_all.py` with the run mapping used in Section 8; the output of that command is `outputs/eval_final/evaluation.json` and `outputs/eval_final/evaluation.md`.
- Multi-model comparison panels are reproduced by `scripts/make_visual_comparison_panel.py`; the figures in this report are stored under `outputs/report_figures/`.

References

- [1] Anonymous authors. “Learning 3D Persistent Embodied World Models.” Reference paper, 2025.
- [2] Greff, Klaus, et al. “Kubric: a scalable dataset generator.” *CVPR*, 2022.
- [3] Shi, Xingjian, et al. “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting.” *NeurIPS*, 2015.
- [4] Ballas, Nicolas, et al. “Delving Deeper into Convolutional Networks for Learning Video Representations.” *ICLR*, 2016.
- [5] Ho, Jonathan, et al. “Video Diffusion Models.” *NeurIPS*, 2022.

- [6] Kendall, Alex, and Yarin Gal. “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” *NeurIPS*, 2017.

normal_case: 00006.npz @ frame 0
Median example from all validation windows.

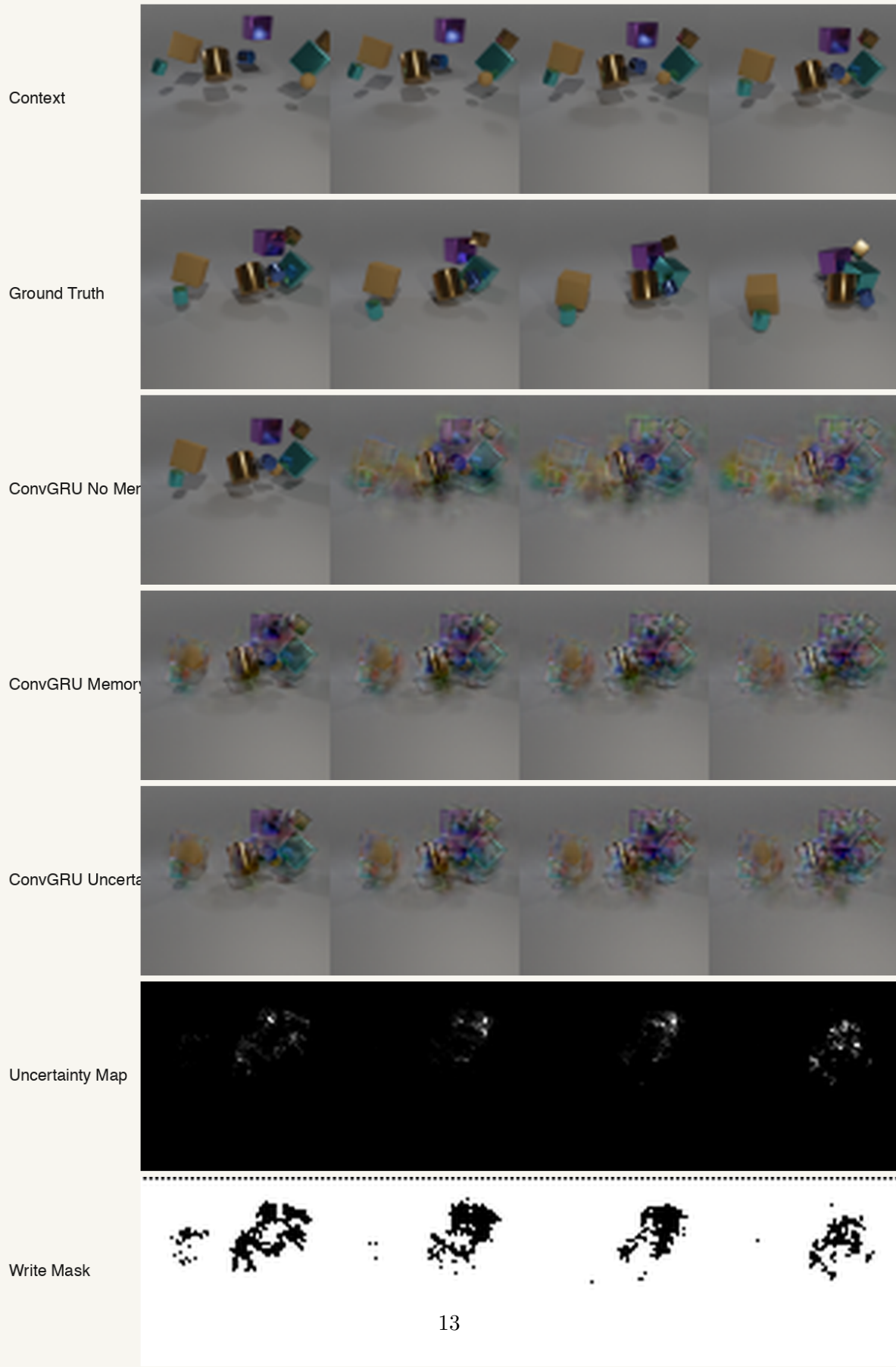
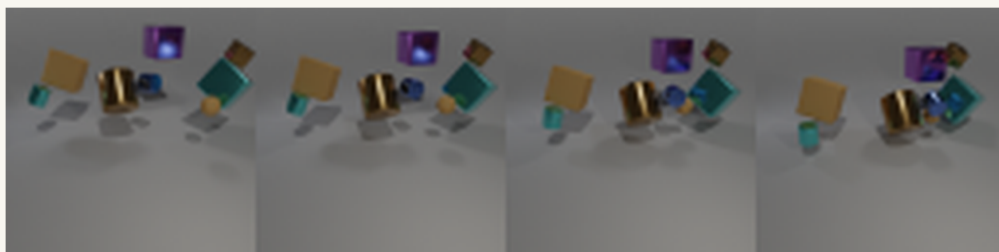


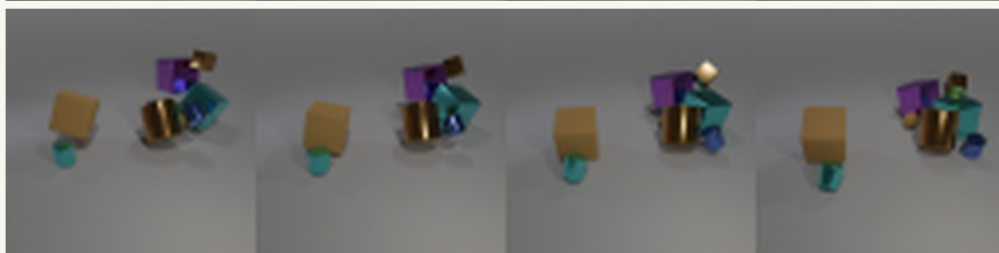
Figure 2: Multi-model comparison on the median validation case. The memory and uncertainty

high_motion: 00006.npz @ frame 1
Representative high-motion example.

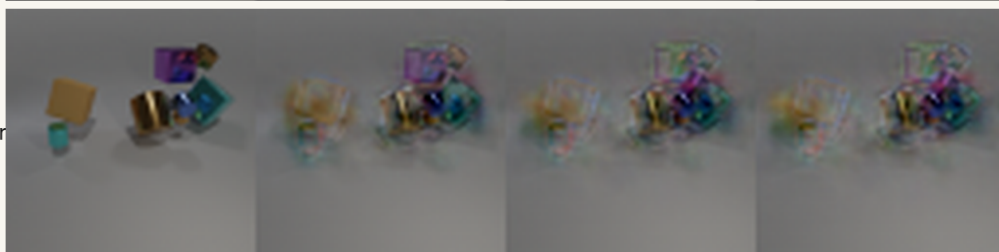
Context



Ground Truth



ConvGRU No Mem



ConvGRU Memory



ConvGRU Uncertainty



Uncertainty Map



Write Mask

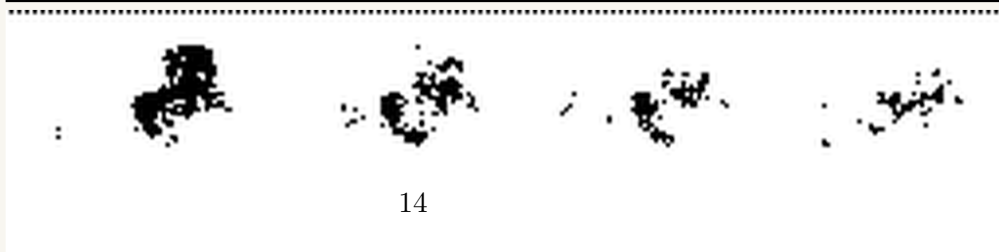


Figure 3: Multi-model comparison on a high-motion validation case.

occlusion_recovery: 00017.npz @ frame 1
Representative occlusion/reappearance example.

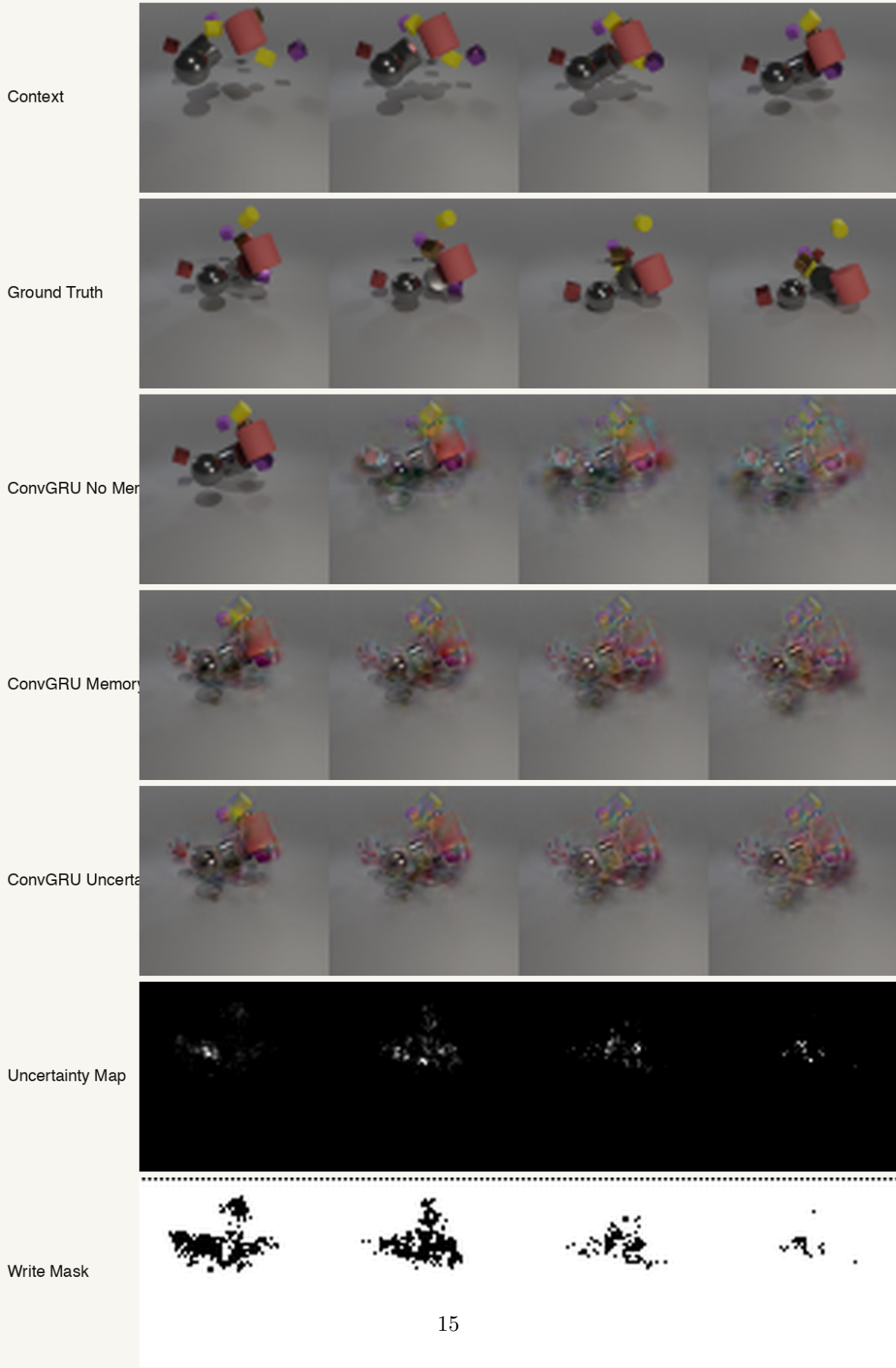
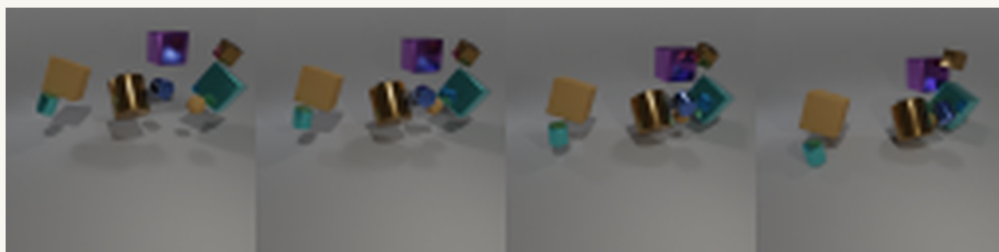


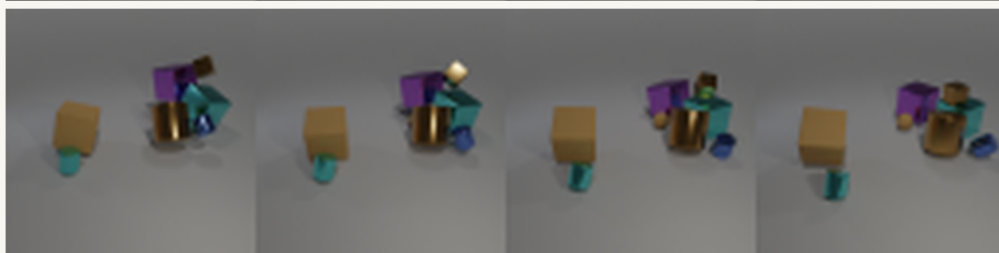
Figure 4: Multi-model comparison on an occlusion-recovery validation case (an object that becomes

high_memory_coverage: 00006.npz @ frame 2
Representative high-memory-coverage example.

Context



Ground Truth



ConvGRU No Mem



ConvGRU Memory



ConvGRU Uncertainty



Uncertainty Map



Write Mask

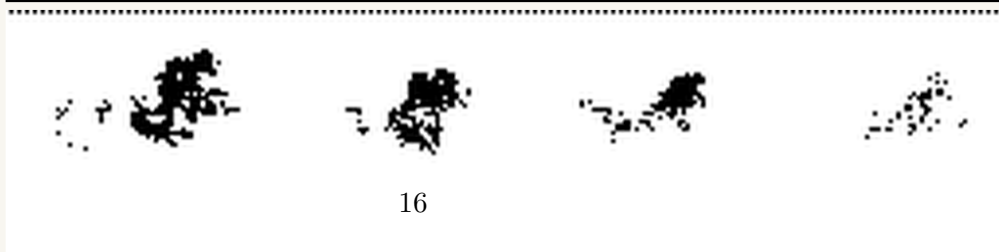


Figure 5: Multi-model comparison on a high-memory-coverage validation case.



Figure 6: Uncertainty diagnostics. Left: predicted per-pixel confidence (brighter = more confident). Right: derived write mask (white = allowed to update memory).